# Lab-3.1 Hồi quy tuyến tính đơn giản

## 0.1 Simple Linear Regression - Hồi quy tuyến tính đơn giản

Hồi quy tuyến tính đơn giản là tìm mối quan hệ tốt nhất giữa biến đầu vào x (biến độc lập) và biến kỳ vọng y (biến phụ thuộc). Mối quan hệ tuyến tính giữa hai biến này có thể được biểu diễn bằng một đường thẳng gọi là đường hồi quy.

---

Simple Linear Regression is finding the best relationship between the input variable x (independent variable) and the expected variable y (dependent variable). The linear relationship between these two variables can be represented by a straight line called regression line.

Công thức: $y = b0 + b1x$

Các thuật ngữ biểu diễn cái gì? - $y$ là phản hồi hoặc biến mục tiêu - $x$ là đặc điểm - $b1$ là hệ số của x - $b0$ là giá trị cắt

---

Formula :- $y = b0 + b1x$

What do terms represent? - $y$ is the response or the target variable - $x$ is the feature - $b1$ is the coefficient of x - $b0$ is the intercept

**Ước tính Hệ số Mô hình ("Học tập")** Các hệ số được ước tính bằng cách sử dụng **tiêu chuẩn bình phương nhỏ nhất**, tức là, phải tính toán đường phù hợp nhất để giảm thiểu **tổng bình phương phần dư** (hoặc "tổng bình phương lỗi").

---

**Estimating ("Learning") Model Coefficients** The coefficients are estimated using the **least-squares criterion**, i.e., the best fit line has to be calculated that minimizes the **sum of squared residuals** (or "sum of squared errors").

## 0.2 Chia mã thành các bước để hiểu rõ hơn:

1. Tải tập dữ liệu.
2. Hình dung dữ liệu.
3. Đào tạo Mô hình hồi quy tuyến tính đơn giản.

---

## 0.3 Dividing the code into steps for better understanding:

1. Load the dataset.
2. Visualize the data.
3. Training Simple Linear Regression Model.

### 0.4 Step1 : Load Dataset

[3]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

[4]:
```
dataset = pd.read_csv('/content/Salary_Data.csv')
```

[5]:
```
dataset.head()
```

[5]:

| | YearsExperience | Salary |
|---|---|---|
| 0 | 1.1 | 39343 |
| 1 | 1.3 | 46205 |
| 2 | 1.5 | 37731 |
| 3 | 2.0 | 43525 |
| 4 | 2.2 | 39891 |

[6]:
```
dataset.shape
```

[6]:
```
(30, 2)
```

Here, 'YearsExperience' is the independent variable and 'Salary' is the dependent variable which will be predicted based on the value of 'YearsExperience'.

[7]:
```
X = dataset.drop(['Salary'],axis=True)
y = dataset['Salary']
```

[8]:
```
#Now Split The Data
from sklearn.model_selection import train_test_split
X_train,X_test,y_train,y_test = train_test_split(X,y,test_size=0.
↪2,random_state=42)
```

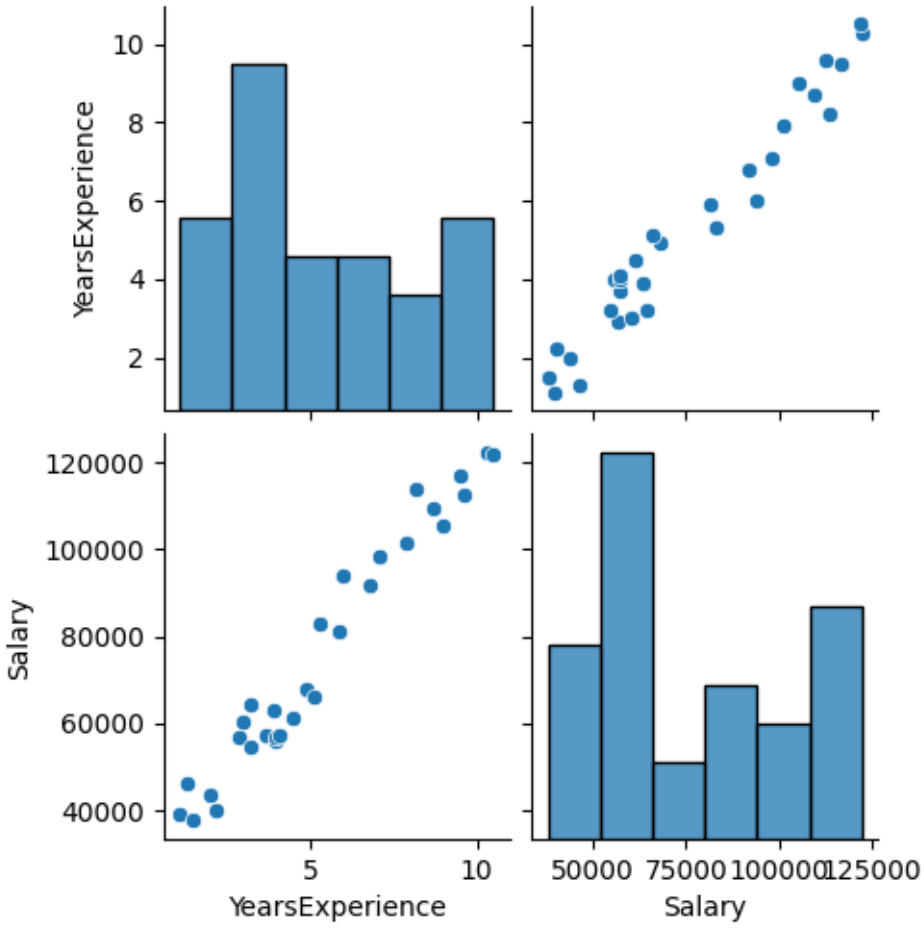
[9]:
```
X_train.shape,X_test.shape,y_train.shape,y_test.shape
```

[9]:
```
((24, 1), (6, 1), (24,), (6,))
```

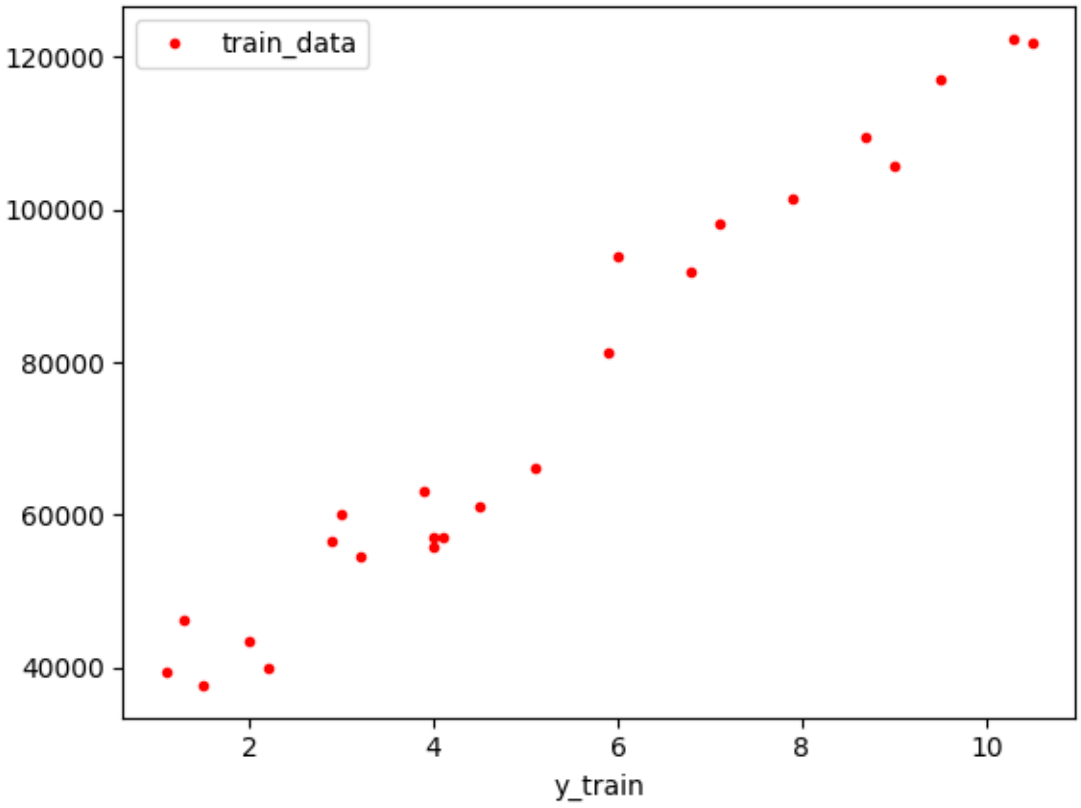
### 0.5 Step 2: Visualize the data.

[10]:
```
# Visualize Whole Data set
sns.pairplot(dataset)
```

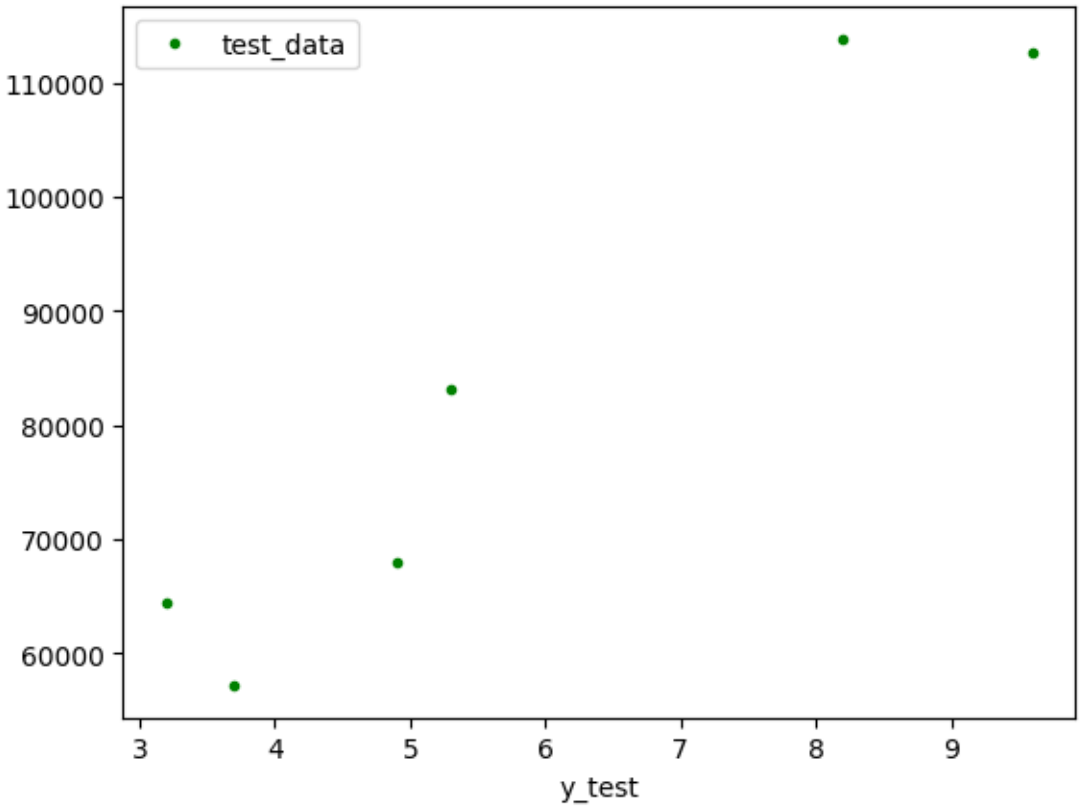
[10]:
```
<seaborn.axisgrid.PairGrid at 0x7d0ba58ee1d0>
```




[11]:
```
# Visualize Only Train data
plt.plot(X_train,y_train,'r.',label='train_data')
plt.xlabel('X_train')
plt.xlabel('y_train')
plt.legend()
plt.show()
```




[12]:
```
# Visualize Only Test data
plt.plot(X_test,y_test,'g.',label='test_data')
plt.xlabel('X_test')
plt.xlabel('y_test')
plt.legend()
plt.show()
```




In The Above three graph clealr shows that the data is linearly deparable so here we use Linear Regression

**Ok, now we hope the dataset is pretty clear by now. We will move to the next step now!**

## 0.6 Step 3 :Training Simple Linear Regression Model.

[13]:
```
from sklearn.linear_model import LinearRegression
regressor = LinearRegression()
regressor.fit(X_train, y_train)
```

[13]:
```
LinearRegression()
```

### 0.6.1 Predicting the Test set results

[14]:
```
y_pred = regressor.predict(X_test).round(1)
```

[15]:
```
calculation = pd.DataFrame(np.c_[y_test,y_pred], columns = ["Original␣
↪Salary","Predict Salary"])
calculation
```

[15]:

| | Original Salary | Predict Salary |
|---|---|---|
| 0 | 112635.0 | 115790.2 |
| 1 | 67938.0 | 71498.3 |
| 2 | 113812.0 | 102596.9 |
| 3 | 83088.0 | 75267.8 |
| 4 | 64445.0 | 55477.8 |
| 5 | 57189.0 | 60189.7 |

[16]:
```
print("Training Accuracy :", regressor.score(X_train, y_train))
print("Testing Accuracy :", regressor.score(X_test, y_test))
```

```
Training Accuracy : 0.9645401573418146
Testing Accuracy : 0.9024461774180497
```

### 0.7 Visualising the Training set results

[17]:
```
plt.scatter(X_train, y_train, color = 'red')
plt.plot(X_train, regressor.predict(X_train), color = 'blue')
plt.title('Salary vs Experience (Train set)')
plt.xlabel('Years of Experience')
plt.ylabel('Salary')
plt.show()
```



### 0.8 Visualising the Training set results

[18]:
```
plt.scatter(X_test, y_test, color = 'red')
plt.plot(X_train, regressor.predict(X_train), color = 'blue')
plt.title('Salary vs Experience (Test set)')
plt.xlabel('Years of Experience')
plt.ylabel('Salary')
plt.show()
```

